Universidad San Carlos de Guatemala

Facultad de ingeniería.

Ingeniería en ciencias y sistemas





Título del Proyecto:

CLOUDGRID DATACENTER

PONDERACIÓN: 25

Horas Aproximadas: 50

Índice

1. Resumen Ejecutivo **3**

2. Competencia que desarrollaremos **3**

3. Objetivos del Aprendizaje **3**

3.1 Objetivo General 3
3.2 Objetivos Específicos 3

4. Enunciado del Proyecto **4**

4.1 Descripción del problema a resolver 4
4.2 Alcance del proyecto 7

5. Desarrollo de Habilidades Blandas **8**

5.1 Proyectos Individuales 8
5.2 Resumen de Puntuaciones 8
5.3 Observaciones 9
5.4 Comentarios Generales 10

1. Resumen Ejecutivo

El centro de procesamiento de datos **CloudGrid DataCenter** enfrenta retos significativos en la administración y monitoreo de su infraestructura tecnológica debido a la dispersión de registros de servidores virtuales (VMs), la asignación de espacios físicos en Racks de servidores, el seguimiento de contratos corporativos de clientes y la gestión oportuna de tiquetes de alerta o incidencias de red. La falta de una plataforma integrada genera fricciones operativas, cuellos de botella en la asignación de capacidad de cómputo y retrasos en la respuesta a fallas críticas del sistema.

Con el propósito de resolver estas ineficiencias de forma integral, se plantea el desarrollo de CloudGrid OS, una aplicación de escritorio con interfaz gráfica desarrollada íntegramente en lenguaje C++. El sistema automatizará el aprovisionamiento de infraestructura mediante la implementación manual de cuatro estructuras de datos dinámicas avanzadas: un Árbol Binario de Búsqueda (BST) para el catálogo ordenado de Máquinas Virtuales; una Matriz Dispersa (Sparse Matrix) para el mapeo ortogonal de Racks y Slots U de altura; una Lista Circular Doblemente Enlazada para el anillo de atención rotativa de tiquetes de incidencia; y una Multilista (Lista Circular de Listas) para agrupar clientes corporativos y sus microservicios o contenedores activos.

Asimismo, la solución integra un motor de visualización basado en Graphviz para renderizar en tiempo real el estado en memoria de las estructuras de datos, ofreciendo a los administradores e ingenieros DevOps un entorno de auditoría visual de alto nivel.

2. Competencia que desarrollaremos

A través de la concepción y programación del sistema CloudGrid DataCenter, se consolidarán competencias avanzadas en el diseño, análisis e implementación de estructuras de datos lineales y no lineales dinámicas en C++ sin el uso de librerías de plantillas estándar (STL) para el manejo de punteros. El estudiante fortalecerá sus habilidades prácticas al modelar sistemas complejos de infraestructura en la nube, garantizando el control riguroso sobre la memoria dinámica, la prevención de fugas de memoria (memory leaks) y la eficiencia algorítmica en operaciones de inserción, búsqueda, recorrido e iteración ortogonal.

Adicionalmente, se desarrollarán capacidades en ingeniería de software modular, diseño de interfaces gráficas funcionales, procesamiento sintáctico de archivos planos de carga masiva (CSV) y generación automatizada de diagramas de grafos mediante archivos de descripción en formato DOT de Graphviz. Todo esto fomenta el pensamiento analítico, la abstracción de problemas reales y el desarrollo de código limpio, mantenible y profesional.

3. Objetivos del Aprendizaje

3.1 Objetivo General

Desarrollar una aplicación de escritorio gráfica en C++ para la gestión integral de un centro de datos Cloud, aplicando estructuras de datos dinámicas hechas a la medida (BST, Matriz Dispersa, Lista Circular Doble y Multilista) para optimizar el inventario de servidores, la asignación de espacio en Racks y el monitoreo de incidencias, integrando reportes gráficos en tiempo real con Graphviz

3.2 Objetivos Específicos

Al finalizar el proyecto, los estudiantes deberán ser capaces de:

- Implementar manualmente en C++ las estructuras de datos dinámicas requeridas, gestionando correctamente nodos, punteros dobles y liberación de memoria.
- Diseñar e implementar una Matriz Dispersa ortogonal para representar la distribución física de Racks y Slots U de altura dentro del Data Center.
- Construir un Árbol Binario de Búsqueda (BST) eficiente para almacenar el catálogo de Servidores Virtuales (VMs) manteniendo un orden estricto por código de identificación e impresión en recorrido Inorden.
- Desarrollar un módulo de carga masiva mediante archivos CSV para la ingesta rápida de inventarios de máquinas virtuales.
- Integrar la herramienta Graphviz para la traducción dinámica de estructuras de memoria a representaciones visuales en formato PNG/PDF.
- Aplicar mejores prácticas de control de versiones con Git/GitHub y documentar exhaustivamente el diseño de software en un manual técnico.

4. Enunciado del Proyecto

4.1 Descripción del problema a resolver

El sistema debe bifurcar su operación en dos perfiles de usuario claramente diferenciados en el formulario de inicio de sesión: **Administrador de Data Center** y **Usuario DevOps / Cliente Corporativo**. A continuación se especifican las funcionalidades obligatorias para cada perfil:

Funciones de Administrador

El administrador del sistema contará con un menú principal en una interfaz gráfica formal que le permitirá realizar las siguientes funciones:

1. Gestionar Servidores Virtuales (VMs) en Catálogo

El administrador podrá registrar individualmente o eliminar máquinas virtuales. Atributos obligatorios: Código único de VM (ej. VM001, VM002), Nombre del Servidor, Sistema Operativo, Memoria RAM (GB), Almacenamiento SSD (GB), Estado (Activa, Inactiva, Mantenimiento), Fecha de Despliegue y Fecha de Expiración. Las VMs se insertan en un Árbol Binario de Búsqueda (BST) ordenado por Código de VM, permitiendo recorridos Inorden.

2. Carga Masiva de Servidores Virtuales (CSV):

Procesar un archivo CSV con laestructura:

CodigoVM,Nombre,SistemaOperativo,RAM_GB,Storage_GB,Estado,FechaDespliegue,

FechaExpiracion. El parser debe validar los tipos de datos e insertar los nodos en el BST.

3. Gestionar Clientes Corporativos y Microservicios

Permite registrar clientes corporativos (Código Cliente, Nombre Empresa, Plan Contratado, Vigencia). Los clientes están vinculados en una Lista Circular Simple principal. Cada cliente posee una Lista Doblemente Enlazada secundaria de Microservicios o Contenedores asignados (ID Contenedor, Nombre Servicio, Puerto, vCPU/RAM).

4. Gestionar Tiquetes de Incidencia / Alertas de Red

Administrar las alertas de red que ingresan en una cola circular doblemente enlazada. Atributos: Número de Tiquete (Autogenerado), Empresa/Cliente, Servidor Afectado, Tipo de Incidencia (CPU 99%, Falla Enlace, Disco Lleno, Kernel Panic), Descripción, Fecha y Estado (Pendiente, En Proceso, Atendida, Rechazada). El admin puede procesar el primer tiquete de la cola, cambiar su estado o rechazarlo.

5. Visualizar Catálogo Completo e Historial de Servidores

Muestra el listado de VMs ordenado por Código de VM indicando su estatus. El sistema debe emitir alertas visuales para servidores cuyo contrato de arrendamiento venza en menos de 7 días.

6. Configurar Racks y Mapa Físico

Permite crear y configurar las dimensiones físicas de un Rack del Data Center (Filas = Racks 1..N, Columnas = Slots U 1..M de altura). Al inicializar un Rack, se genera una Matriz Dispersa ortogonal. El admin puede ver la ocupación, asignar servidores físicos a slots específicos o liberar slots.

Funciones del Módulo Usuario DevOps / Cliente

Los usuarios tendrán acceso a las siguientes funciones:

1. Iniciar Sesión

Autenticación segura mediante correo electrónico y contraseña

2. Consultar Catálogo de VMs

Buscar y visualizar los detalles técnicos de los servidores virtuales registrados en el BST.

3. Reservar Slot en Rack

Seleccionar un Rack y Slot U de altura disponible dentro de la Matriz Dispersa y registrar la reserva a nombre de la empresa.

4. Cancelar Reserva

Liberar un slot en la Matriz Dispersa, retornando su estado a LIBRE.

Área de Reportes

El sistema integrará un módulo especializado que generará de forma automática archivos en formato DOT y ejecutará el binario de Graphviz para producir imágenes PNG de las 4 estructuras de datos dinámicas en tiempo real:

Reporte 1: Árbol binario de busqueda (Catálogo de VMs)

Representa la jerarquía de servidores virtuales ordenados por Código de VM. Cada nodo muestra el Código, Nombre, SO, RAM/Storage y un código de colores (Verde: Activa, Amarillo: Mantenimiento, Rojo: Inactiva).

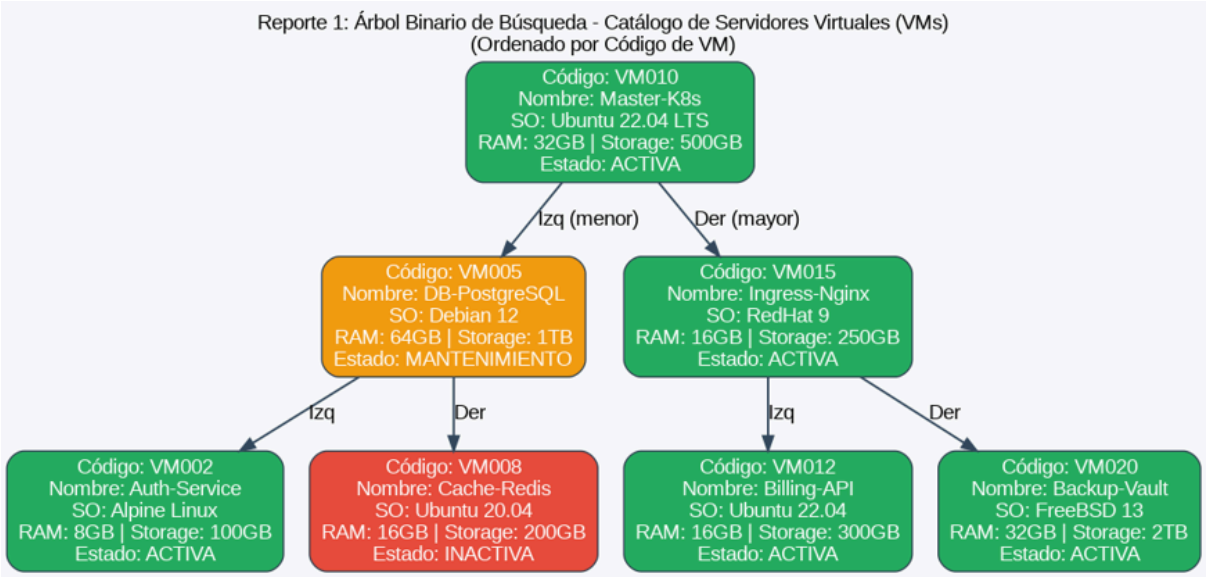


Figura 1: Estructura de Árbol BST para el Catálogo de VMs.

Reporte 2: Lista Circular Doblemente Enlazada (Tiquetes de Incidencia)

Muestra el anillo rotativo de atención de tiquetes de alerta. Cada nodo Mrecord muestra los punteros anterior/siguiente, el cliente, la falla y el estado con flechas bidireccionales en circuito cerrado.

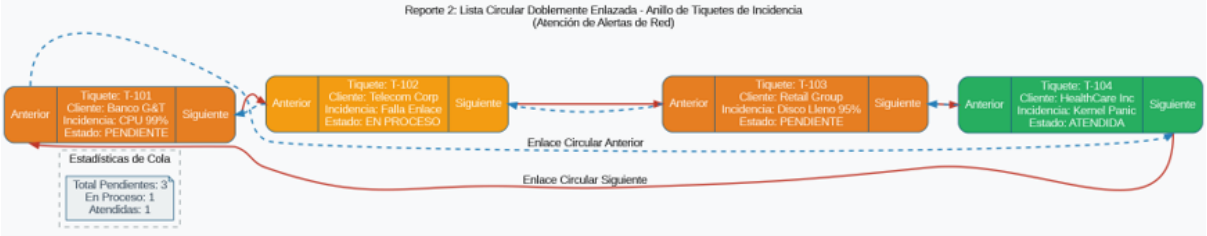


Figura 2: Anillo Circular Doblemente Enlazado de Tiquetes de Incidencia

Reporte 3: Multilista / Lista Circular de Listas (Clientes y Contenedores)

Visualiza la lista principal circular horizontal con los clientes corporativos y sus listas secundarias verticales de microservicios o contenedores desplegados.

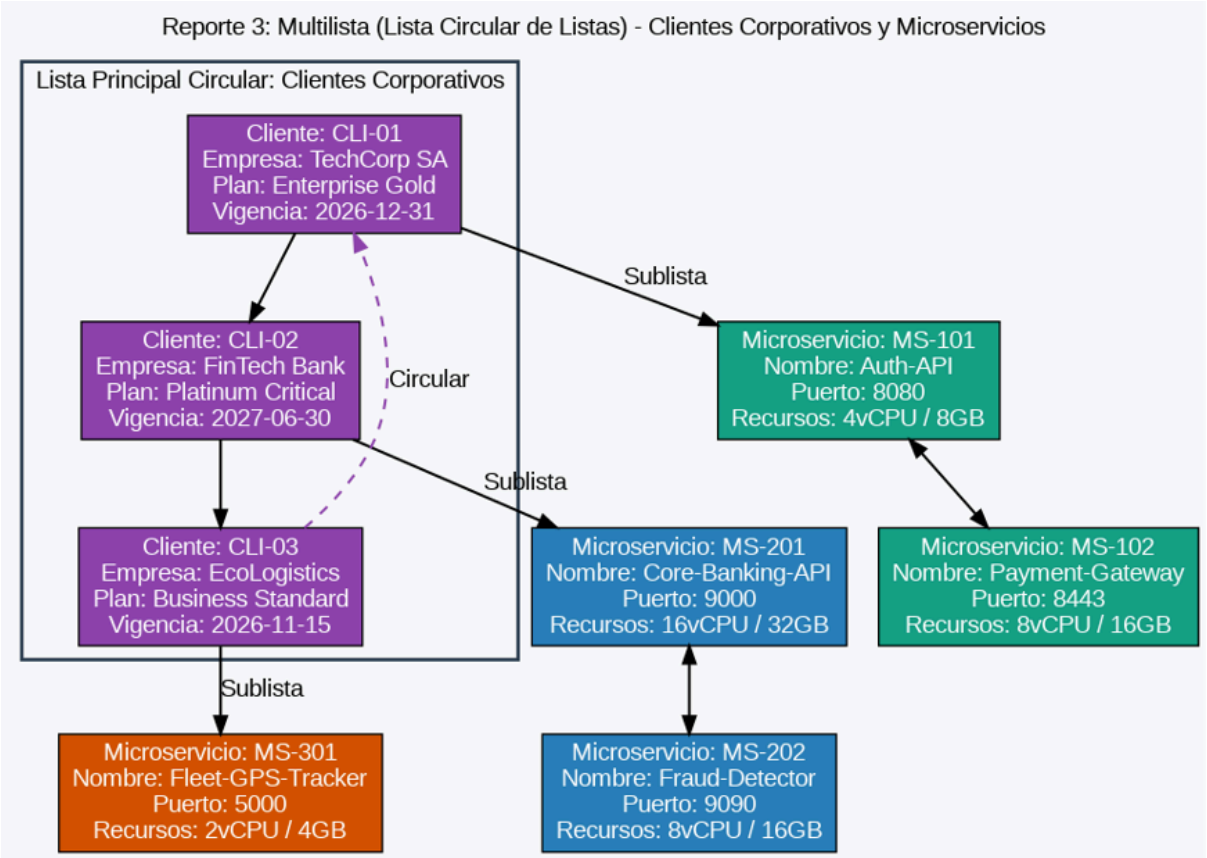


Figura 3: Estructura de Multilista para Clientes Corporativos y Microservicios.

Reporte 4: Matriz Dispersa (Mapa Ortogonal de Racks y Slots U)

Visualiza la matriz ortogonal de ocupación del Data Center. Incluye nodos cabecera de Racks (Filas) y Slots U (Columnas), y nodos internos conectados horizontal y verticalmente con punteros dobles.

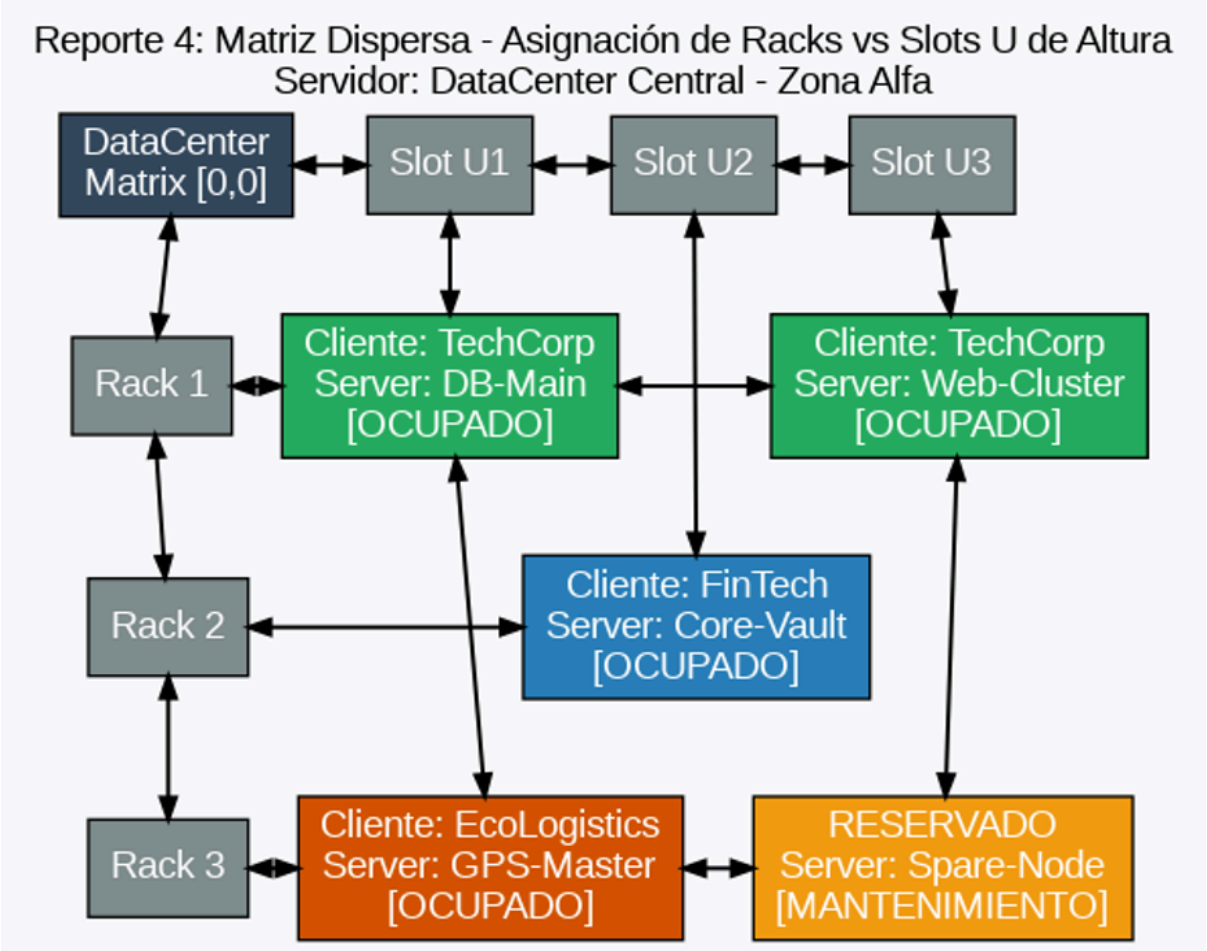


Figura 4: Matriz Dispersa Ortogonal para Asignación de Racks vs Slots U.

4.2 Alcance del proyecto

Las estructuras de datos deben ser desarrolladas completamente por los estudiantes sin utilizar librerías o módulos predefinidos en c++ que implementen directamente estas estructuras. Toda la lógica de enlazado, inserción, eliminación y recorrido debe ser manual.

Los reportes gráficos son esenciales para la evaluación. Si no se presenta el reporte de alguna estructura, se anularán los puntos correspondientes. El código debe ser original, individual y debidamente comentado

4.2 Requerimiento técnicos

Realizar la aplicación en c++

El sistema se desarrollará completamente en c++, implementando una aplicación con interfaz que gestione todas las funcionalidades descritas. Se utilizarán las capacidades nativas de c++ para el manejo de archivos CSV y procesamiento de texto. El código deberá ser modular, con funciones claramente definidas para cada operación del sistema.

Realizar los reportes en Graphviz

Todos los reportes gráficos deberán generarse utilizando exclusivamente Graphviz. El sistema c++ creará archivos DOT con la descripción de los grafos y ejecutará los comandos necesarios para generar las imágenes en formato PNG o PDF. Los reportes se actualizarán automáticamente cada vez que se modifiquen las estructuras de datos, permitiendo una visualización en tiempo real del estado del sistema.

5. Desarrollo de Habilidades Blandas

Para complementar el desarrollo técnico, esta sección se centra en las habilidades blandas que los estudiantes deberán mejorar a lo largo del proyecto, como la comunicación, el liderazgo, y la colaboración en equipo.

5.1 Proyectos Individuales

Los proyectos individuales permiten a los estudiantes desarrollar autonomía y responsabilidad sobre su propio trabajo. En este tipo de proyectos, cada estudiante debe gestionar su tiempo, investigar, planificar y ejecutar todas las fases del proyecto de manera independiente.

5.1.1 Autogestión del Tiempo

Los estudiantes deben crear un cronograma personal para cumplir con los plazos establecidos. Esto les ayuda a mejorar su disciplina y capacidad de priorización, habilidades esenciales en cualquier entorno profesional.

5.1.2 Responsabilidad y Compromiso

En un proyecto individual, el estudiante asume la totalidad de las responsabilidades, desde la investigación hasta la entrega final. Esto fomenta el sentido de compromiso y permite una mayor personalización en la solución del problema planteado.

5.1.3 Resolución de Problemas

Trabajar de manera independiente impulsa a los estudiantes a buscar soluciones de forma creativa y a enfrentarse a los desafíos sin depender del apoyo constante de otros. Esto fortalece su capacidad para resolver problemas de manera autónoma.

5.1.4 Reflexión Personal

Al concluir el proyecto, el estudiante realiza una autoevaluación, reflexionando sobre sus decisiones, lo aprendido y las áreas en las que podría mejorar. Esta práctica promueve el desarrollo continuo y el autoaprendizaje.

5.2 Resumen de Puntuaciones

Área	Puntos Totales	Puntos Obtenidos
1. Implementación de estructuras de datos	**40**	
Árbol Binario de Búsqueda (BST) para Servidores VMs	10	
Multilista (Lista Circular de Listas) para Clientes y Microservicios	9	
Lista Circular Doblemente Enlazada para Tiquetes de Incidencia	9	
Matriz Dispersa Ortogonal para Racks y Slots U	12	
2. Funcionalidades del sistema	**31**	
Funciones del Módulo Administrador (Carga CSV, Altas, Bajas, Racks)	19	
Funciones del Módulo Usuario DevOps (Consultas, Reservas, Tiquetes)	12	
3. Reporte gráficos con Graphviz	**18**	
Reporte de Árbol Binario de Búsqueda (BST)	5	
Reporte de Lista Circular Doblemente Enlazada	3	
Reporte de Multilista / Lista Circular de Listas	2	
Reporte de Matriz Dispersa Ortogonal	8	
4. Manual técnico y preguntas	**11**	
TOTAL	**100**	

5.3 Observaciones

- El lenguaje de programación para esta fase será c++.
- La herramienta de desarrollo de reportes gráficos será exclusivamente Graphviz.
- La entrega se realizará por medio de Github. El nombre del repositorio debe ser EDD_1S2027_PY_#carnet (Ej: EDD_1S2027_PY_202102771), donde se creará una carpeta con el nombre EDD_Fase1.
- Por medio de UEDI se hará entrega del link del repositorio.
- Deberá agregar a su respectivo auxiliar a su reporte como colaborador:
    - user → Sección B
    - user → Sección C
    - user → sección A
- Los repositorios deben mantenerse en privado para evitar copias de código entre estudiantes.
- Se debe realizar el manual técnico en formato Markdown dentro de una carpeta llamada documentación, dentro de la carpeta de la respectiva fase, explicando la implementación de cada estructura y las decisiones de diseño tomadas.
- Todas las dudas durante el proceso de desarrollo deberán realizarse por medio de los foros de UEDI de la respectiva sección.
- Las copias detectadas serán penalizadas con calificación de 0 puntos y reportadas a la Escuela de Ciencias y Sistemas.
- Se debe de realizar commits frecuentes en Github para demostrar el avance progresivo del proyecto.
- El código debe estar debidamente comentado y seguir buenas prácticas de programación.
- Se debe incluir un archivo de prueba CSV con datos de ejemplo para demostrar la funcionalidad de carga masiva.

FECHA DE ENTREGA: xx de xx del 2027